[illegible]

Anti-Interference-2D [illegible]2D[illegible]

[illegible]

1. [illegible]

1.1 [illegible]

[illegible]

[illegible]
- [illegible]
- [illegible]
- [illegible]1-12m[illegible]
- [illegible]

1.2 [illegible]

[illegible]	[illegible]	[illegible]
[illegible]	≥95%	7[illegible]
[illegible]	≤0.2mm	[illegible]
[illegible]	≥100m/min	[illegible]
[illegible]	24/7[illegible]	[illegible]
[illegible]	[illegible]	[illegible]

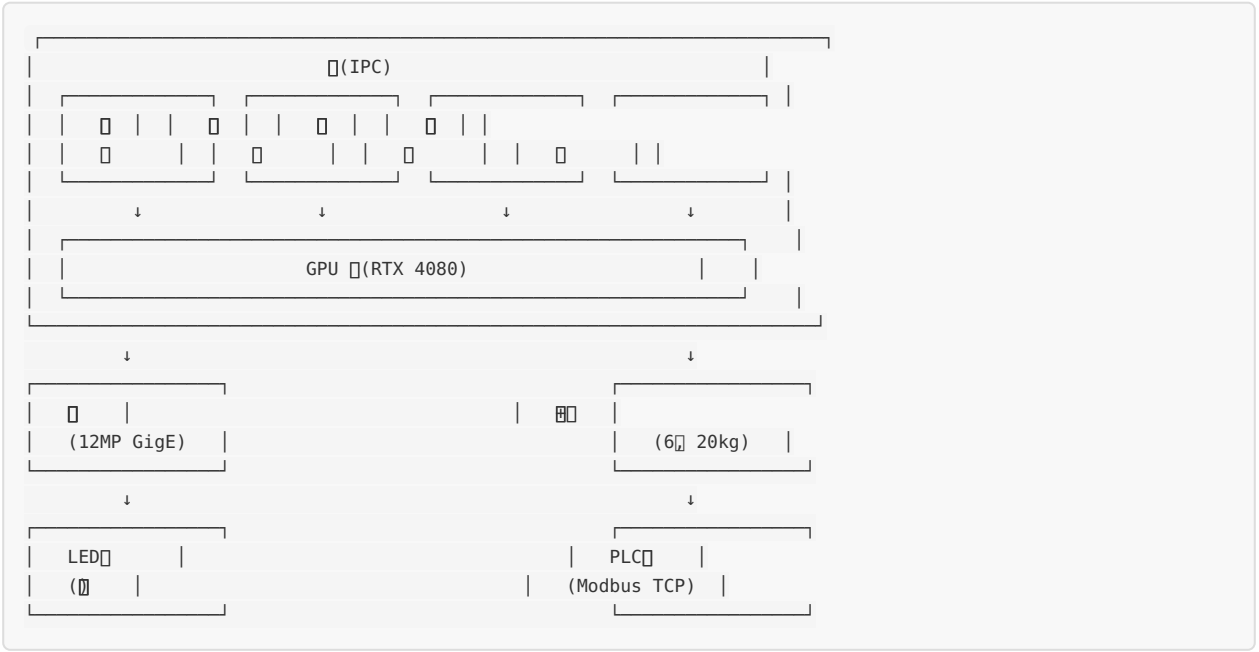
1.3 [illegible]

[illegible]"[illegible] + [illegible] + [illegible]"[illegible]

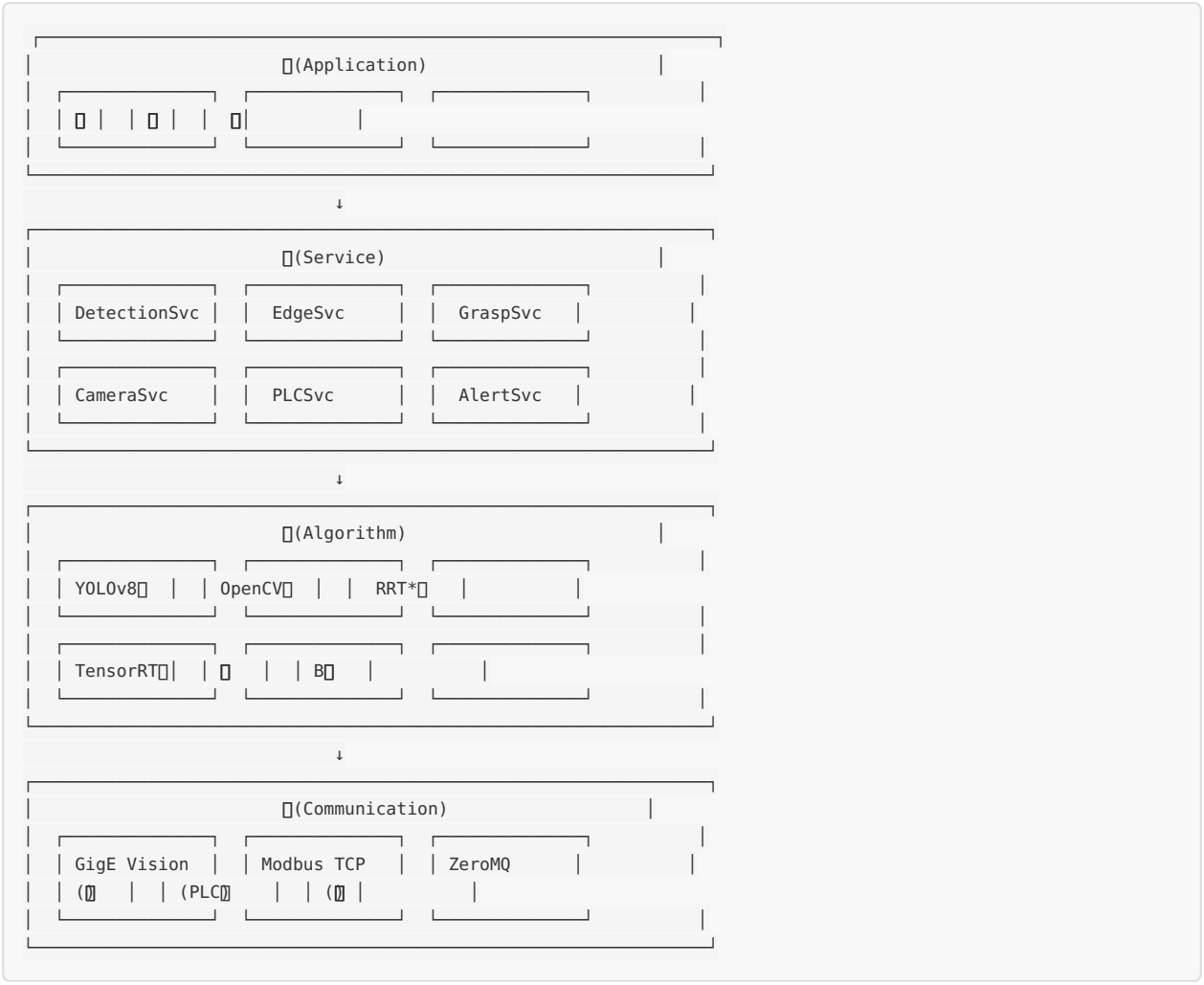
[illegible]	[illegible]	[illegible]
[illegible]	[illegible] + HDR	[illegible]
[illegible]	YOLOv8 + [illegible]	[illegible]
[illegible]	ResNet-18 + [illegible]	[illegible]
[illegible]	RRT* + [illegible]	[illegible]
[illegible]	TensorRT + [illegible]	[illegible]

2. □□□□□□

2.1 □□□□



2.2 □□□□


□(Application)
□(Service)
DetectionSvc
EdgeSvc
GraspSvc
CameraSvc
PLCSvc
AlertSvc
□(Algorithm)
YOLOv8□
OpenCV□
RRT*□
TensorRT□
B□
□(Communication)
GigE Vision
Modbus TCP
ZeroMQ

2.3 □□□□

```
□
          ↓
┌───────────────────────────────────────────┐
│   1. □                             │
│       - □                             │
│       - HDR□                             │
│       - RGB+□                         │
└───────────────────────────────────────────┘
          ↓
┌───────────────────────────────────────────┐
│   2. □                                   │
│       - YOLOv8□(<20ms)                      │
│       - □              │
│       - □                             │
└───────────────────────────────────────────┘
          ↓
┌───────────────────────────────────────────┐
│   3. □                                   │
│       - Canny + □                      │
│       - □                      │
│       - □                      │
└───────────────────────────────────────────┘
          ↓
┌───────────────────────────────────────────┐
│   4. □                                   │
│       - □                                 │
│       - RRT*□                             │
│       - □                             │
│       - □                              │
└───────────────────────────────────────────┘
          ↓
┌───────────────────────────────────────────┐
│   5. □                                 │
│       - PLC□                                │
│       - □                                 │
│       - □                                   │
└───────────────────────────────────────────┘
```

---

3. □□□□□□

3.1 □□□□□□□

□□

□□□□□

□□□□□□□□□□□□□□□

- □□□□□□□□□□□□□□□80%□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□

□□□□□□□□□□□□

- □□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□

3.2 □□□□□□□□□

□□	□□	□□
□□□	□□□□	□□ 100×100mm□PD>95%□□□□□>1000:1
□□□	□□□□	□□□□□□□□□□□□□90°±2°□
□□	LED□□□	□□ 200W□□□ 5600K□□□□>95%
□□□□□	□□□□□	□□□□ 0-100%□□□□□

□□□□□□□□

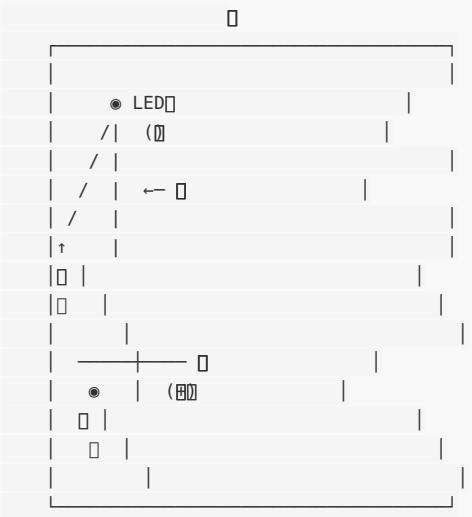
```
def optimize_polarizer_angle(incident_angle, defect_type):
    """
    □

    Args:
        incident_angle: □
        defect_type: □

    Returns:
        optimal_angles: (□, □)
    """
    # □
    brewster_angle = 70  # □

    # □
    if abs(incident_angle - brewster_angle) < 10:
        # □
        polarization_angle = 90
    else:
        # □
        polarization_angle = 85

    return polarization_angle, polarization_angle + 90
```

3.3 HDR□□□□

□□□□□

```
 import cv2
import numpy as np

class HDRImaging:
    """□"""

    def __init__(self, num_exposures=5):
        self.num_exposures = num_exposures
        self.exposure_times = [1/500, 1/250, 1/125, 1/60, 1/30]  # □

    def capture_hdr(self, camera):
        """□HDR□"""
        images = []
        for exp_time in self.exposure_times:
            # □
            camera.set_exposure(exp_time)
            # □
            time.sleep(0.05)
            # □
            img = camera.grab()
            images.append(img)

        return images

    def merge_hdr(self, images, exposure_times):
        """
        □HDR□

        Args:
            images: □[LDR1, LDR2, ...]
            exposure_times: □

        Returns:
            hdr: HDR□(32□)
        """
        # □TV□
        merge = cv2.createMergeMertens()
        merged = merge.process(images)

        # □0,1]
        merged = np.clip(merged, 0, 1)

        return merged

    def tone_mapping(self, hdr, method='Drago'):
        """
        □

        Args:
            hdr: HDR□
            method: □('Drago', 'Reinhard', 'Mantiuk')

        Returns:
            ldr: □DR□
        """
        if method == 'Drago':
            tonemap = cv2.createTonemapDrago(1.0, 0.3)
        elif method == 'Reinhard':
            tonemap = cv2.createTonemapReinhard(1.0, 0.5)
        else:  # Mantiuk
            tonemap = cv2.createTonemapMantiuk(2.2, 0.85, 1.2)

        ldr = tonemap.process(hdr)
        ldr = np.clip(ldr * 255, 0, 255).astype(np.uint8)

        return ldr
```

3.4 □□□□□□□



---

4. □□□□□□

4.1 □□□□□□□

□□ID	□□□□	□□	□□□	□□□□
D01	□□	□□□□□□□	□	□
D02	□□	□□□□	□	□□
D03	□□	□□□□	□	□□
D04	□□	□□□□□	□	□
D05	□□	□□□□□□	□	□□
D06	□□	□□□□□	□□	□□
D07	□□□□	□□□□□□	□	□□

4.2 YOLOv8□□□□

□□□□□

```yaml
 # model_config.yaml
model:
  name: yolov8m
  pretrained: yolov8m.pt
  input_size: 640
  channels: 3

training:
  epochs: 300
  batch_size: 16
  optimizer: AdamW
  lr0: 0.001
  lrf: 0.01
  weight_decay: 0.0005
  warmup_epochs: 3
  warmup_momentum: 0.8

data_augmentation:
  hsv_h: 0.015
  hsv_s: 0.7
  hsv_v: 0.4
  degrees: 10
  translate: 0.1
  scale: 0.5
  fliplr: 0.5
  mosaic: 1.0
  mixup: 0.1

detection:
  conf_threshold: 0.5
  iou_threshold: 0.4
  max_det: 100
```

□□□□□

```
class DefectDetectionPipeline:
    """□"""

    def __init__(self, model_path, use_tensorrt=True):
        self.use_tensorrt = use_tensorrt

        if use_tensorrt:
            self.engine = TensorRTInfer(model_path.replace('.pt', '.engine'))
        else:
            self.model = YOLO(model_path)
            self.model.to('cuda')

    @torch.no_grad()
    def detect(self, image):
        """
        □

        Args:
            image: (H, W, 3) RGB□uint8

        Returns:
            detections: [{'bbox': [x1,y1,x2,y2], 'conf': 0.95, 'class': 0, 'name': '□'}, ...]
        """
        # □
        input_tensor = self.preprocess(image)

        # □
        if self.use_tensorrt:
            outputs = self.engine.inference(input_tensor)
            detections = self.postprocess_tensorrt(outputs)
        else:
            results = self.model.predict(input_tensor, verbose=False)
            detections = self.postprocess_yolo(results)

        return detections

    def preprocess(self, image):
        """□"""
        # □
        img = image.astype(np.float32) / 255.0

        # □
        img = cv2.resize(img, (640, 640))

        # □BGR->RGB
        img = img[:, :, ::-1]

        # HWC->NCHW
        img = np.transpose(img, (2, 0, 1))
        img = np.expand_dims(img, 0)

        return img

    def postprocess_tensorrt(self, outputs):
        """□TensorRT□"""
        # □OLO□
        # outputs: (1, 84, 8400) [obj, x, y, w, h, cls...]
        outputs = outputs[0]  # (84, 8400)

        # □(8400, 84)
        outputs = outputs.T

        # □
        detections = []
        for det in outputs:
            obj_conf = det[4]
```

```
            if obj_conf < 0.5:
                continue

            cls_conf = det[5:]
            cls_id = np.argmax(cls_conf)
            cls_conf = cls_conf[cls_id]

            if cls_conf < 0.5:
                continue

            # □
            x, y, w, h = det[:4]
            x1 = x - w/2
            y1 = y - h/2
            x2 = x + w/2
            y2 = y + h/2

            detections.append({
                'bbox': [x1, y1, x2, y2],
                'conf': float(obj_conf * cls_conf),
                'class': int(cls_id),
                'name': self.class_names[cls_id]
            })

        # NMS
        detections = self.nms(detections)

        return detections

    def nms(self, detections, iou_threshold=0.4):
        """□"""
        if not detections:
            return []

        boxes = np.array([d['bbox'] for d in detections])
        scores = np.array([d['conf'] for d in detections])

        # NMS
        indices = cv2.dnn.NMSBoxes(boxes.tolist(), scores.tolist(),
                                   0.5, iou_threshold)

        return [detections[i] for i in indices]
```

4.3 □□□□□□□

□□□□□□□□□□□□

```
class GenerativeAugmentation:
    """□□"""

    def __init__(self, diffusion_model_path):
        self.diffusion = DiffusedModel(diffusion_model_path)

    def generate_defect_samples(self, defect_type, num_samples=100):
        """
        □

        Args:
            defect_type: □
            num_samples: □

        Returns:
            images: □
        """
        prompts = {
            '□': 'steel plate with scratch defect, surface scratch, linear mark',
            '□': 'steel plate with pit, surface depression, dent',
            '□': 'steel plate with rust, corrosion, oxidized surface',
            '□': 'steel plate with pitting, small holes, surface pitting',
            '□': 'steel plate with inclusion, foreign material embedded',
            '□': 'steel plate with crack, crack line, fracture',
        }

        prompt = prompts.get(defect_type, '')
        images = self.diffusion.generate(prompt, num_samples=num_samples)

        return images

    def augment_training_data(self, original_images, defect_masks, num_augmented=1000):
        """
        □

        Args:
            original_images: □
            defect_masks: □
            num_augmented: □

        Returns:
            augmented_images: □
            augmented_labels: □
        """
        augmented = []

        for _ in range(num_augmented):
            # □
            idx = np.random.randint(0, len(original_images))
            img = original_images[idx]
            mask = defect_masks[idx]

            # □
            aug_type = np.random.choice(['cutout', 'paste', 'style_transfer', 'mixup'])

            if aug_type == 'cutout':
                # □
                aug_img = self.cutout_augmentation(img, mask)
            elif aug_type == 'paste':
                # □
                aug_img = self.paste_defect(img, mask)
            elif aug_type == 'style_transfer':
                # □
                aug_img = self.style_transfer(img, defect_type)
            else:
                # MixUp
```

```
                aug_img = self.mixup_augmentation(img, mask)

            augmented.append(aug_img)

        return augmented

    def cutout_augmentation(self, image, mask, num_holes=3):
        """Cutout□"""
        aug_img = image.copy()
        h, w = mask.shape[:2]

        for _ in range(num_holes):
            # □
            x = np.random.randint(0, w)
            y = np.random.randint(0, h)

            # □
            size = np.random.randint(10, 50)

            # □
            x1 = max(0, x - size//2)
            y1 = max(0, y - size//2)
            x2 = min(w, x + size//2)
            y2 = min(h, y + size//2)

            aug_img[y1:y2, x1:x2] = np.random.randint(0, 256, (y2-y1, x2-x1, 3), dtype=np.uint8)

        return aug_img
```

---

5. □□□□□□

5.1 □□□□□□

```
□
  ↓
┌──────────────────┐
│ □     │ □        │
└──────────────────┘
  ↓
┌──────────────────┐
│ □anny     │ □    │
└──────────────────┘
  ↓
┌──────────────────┐
│ □   │ □          │
└──────────────────┘
  ↓
┌──────────────────┐
│ □       │ □      │
└──────────────────┘
  ↓
┌──────────────────┐
│ □     │ 0.1□     │
└──────────────────┘
  ↓
□
```

5.2 □□□□□□

```
class SteelEdgeLocator:
    """□□"""

    def __init__(self):
        self.edge_detector = EdgeDetector()
        self.subpixel_detector = SubpixelEdgeDetector()

    def locate_edge(self, image, defect_regions=None):
        """
        □

        Args:
            image: RGB□
            defect_regions: □

        Returns:
            edges: □[[x, y], ...]
            edge_lines: □
        """
        # □
        gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

        # □
        denoised = cv2.fastNlMeansDenoising(gray, h=10)

        # □
        processed = self.polarized_processing(denoised)

        # □
        edges = self.edge_detector.adaptive_canny(processed)

        # □
        edges = self.morphological_processing(edges)

        # □
        if defect_regions:
            edges = self.mask_defect_regions(edges, defect_regions)

        # □
        skeleton = self.extract_skeleton(edges)

        # □
        refined_edges = self.subpixel_refinement(skeleton)

        # □
        edge_lines = self.fit_edge_lines(refined_edges)

        return refined_edges, edge_lines

    def polarized_processing(self, gray_image):
        """□□"""
        # □
        # □

        # □
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        enhanced = clahe.apply(gray_image)

        return enhanced

    def adaptive_canny(self, gray_image):
        """□anny□"""
        # □
        sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ks =3)
        magnitude = np.sqrt(sobelx**2 + sobely**2)
```

```
        # □
        high_thresh = np.percentile(magnitude, 85)
        low_thresh = high_thresh * 0.4

        # □
        edges = cv2.Canny(gray_image, low_thresh, high_thresh)

        return edges

    def morphological_processing(self, edges):
        """□"""
        kernel = np.ones((3, 3), np.uint8)

        # □
        closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=2)

        # □
        opened = cv2.morphologyEx(closed, cv2.MORPH_OPEN, kernel, iterations=1)

        return opened

    def extract_skeleton(self, binary_image):
        """□"""
        skeleton = np.zeros_like(binary_image)
        element = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))

        temp = binary_image.copy()
        while True:
            eroded = cv2.erode(temp, element)
            opened = cv2.dilate(eroded, element)
            temp_diff = cv2.subtract(temp, opened)
            skeleton = cv2.bitwise_or(skeleton, temp_diff)
            temp = eroded.copy()

            if cv2.countNonZero(temp) == 0:
                break

        return skeleton

    def subpixel_refinement(self, skeleton):
        """□"""
        # □
        points = np.where(skeleton > 0)
        points = np.stack([points[1], points[0]], axis=1)  # (x, y)

        if len(points) == 0:
            return []

        # □
        criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_COUNT, 30, 0.1)
        points_float = points.astype(np.float32)
        refined = cv2.cornerSubPix(
            self.gray_image if hasattr(self, 'gray_image') else np.zeros_like(skeleton),
            points_float,
            (7, 7),
            (-1, -1),
            criteria
        )

        return refined.tolist() if refined is not None else points.tolist()

    def fit_edge_lines(self, edge_points):
        """□"""
        if len(edge_points) < 10:
            return []
```

```
        points = np.array(edge_points, dtype=np.float32)

        # 髓ANSAC
        [vx, vy, x, y] = cv2.fitLine(points, cv2.DIST_L2, 0, 0.01, 0.01)

        # 鬱ngle distance
        angle = np.arctan2(vy, vx)[0] * 180 / np.pi
        cos_angle = np.cos(np.arctan2(vy, vx)[0])
        sin_angle = np.sin(np.arctan2(vy, vx)[0])
        distance = abs(x * sin_angle - y * cos_angle)

        return {
            'angle': float(angle),
            'distance': float(distance),
            'point': (float(x), float(y)),
            'direction': (float(vx), float(vy))
        }
```

5.3 定位精度保障

误差来源	影响大小	补偿措施
相机标定	±0.1mm	定期标定内参、外参
镜头畸变	±0.05mm	畸变校正模型
运动模糊	±0.2mm	全局快门+适当曝光
边缘检测	±0.3像素	亚像素算法
安装误差	±0.1mm	工装定位
温度漂移	±0.05mm	温度补偿

---

6. 抓取规划方案

6.1 抓取策略

抓取点选择原则：

原则	说明	优先级
安全性	避开缺陷区域	必须
可达性	在机械臂工作空间内	必须
稳定性	重心稳定，不易滑落	高
效率	路径短、时间省	中
最小应力	避免应力集中	中

抓取点计算：

```
class GraspPointSelector:
    """□"""

    def __init__(self, robot_workspace, grasp_offset=50):
        """
        Args:
            robot_workspace: □
            grasp_offset: □(mm)
        """
        self.workspace = robot_workspace
        self.grasp_offset = grasp_offset

    def select_grasp_points(self, steel_contour, defect_regions):
        """
        □

        Args:
            steel_contour: □
            defect_regions: □

        Returns:
            grasp_points: □[{'point': (x,y), 'angle': θ, 'quality': score}, ...]
        """
        # □
        x, y, w, h = cv2.boundingRect(steel_contour)

        # □
        candidates = self.generate_candidates(x, y, w, h, steel_contour)

        # □
        scored_points = []
        for point, angle in candidates:
            # □
            if self.is_in_defect(point, defect_regions):
                continue

            # □
            if not self.is_valid_distance(point, steel_contour):
                continue

            # □
            quality = self.evaluate_grasp_quality(point, angle, steel_contour, defect_regions)

            scored_points.append({
                'point': point,
                'angle': angle,
                'quality': quality
            })

        # □
        scored_points.sort(key=lambda p: p['quality'], reverse=True)

        # N□
        return scored_points[:10]  # □op10

    def generate_candidates(self, x, y, w, h, contour):
        """□"""
        candidates = []

        # □
        for i in range(0, len(contour), 10):
            pt = contour[i][0]

            # □
            if i > 0 and i < len(contour) - 1:
                prev = contour[i-1][0]
```

```
                next_pt = contour[i+1][0]
                dx = next_pt[0] - prev[0]
                dy = next_pt[1] - prev[1]
                angle = np.arctan2(dy, dx)
            else:
                angle = 0

            # □
            candidates.append((tuple(pt), angle))

            # □
            offset_pt = (
                int(pt[0] + self.grasp_offset * np.cos(angle)),
                int(pt[1] + self.grasp_offset * np.sin(angle))
            )
            candidates.append((offset_pt, angle))

        return candidates

    def evaluate_grasp_quality(self, point, angle, contour, defects):
        """□"""
        quality = 1.0

        # □
        min_defect_dist = self.min_distance_to_defects(point, defects)
        if min_defect_dist < 100:
            quality *= 0.5

        # □
        edge_dist = self.distance_to_edge(point, contour)
        if edge_dist < 30:
            quality *= 0.7
        elif edge_dist > 200:
            quality *= 0.8

        # □
        # □

        return quality
```

6.2 □□□□□□

```
class SteelGraspPlanner:
    """□□"""

    def __init__(self, workspace_bounds, obstacles=None):
        """
        Args:
            workspace_bounds: [[x_min, x_max], [y_min, y_max], [z_min, z_max]]
            obstacles: □
        """
        self.workspace = workspace_bounds
        self.obstacles = obstacles or []
        self.rrt_planner = RRTPlanner(space_dim=3, bounds=workspace_bounds)

    def plan_grasp_path(self, start, grasp_point, approach_height=100):
        """
        □

        Args:
            start: □[x, y, z]
            grasp_point: □[x, y, z]
            approach_height: □(mm)

        Returns:
            path: □
            success: □
        """
        # □
        pre_grasp = [grasp_point[0], grasp_point[1], approach_height]  # □
        post_grasp = start  # □

        # □
        path_segments = []

        # 1. □
        segment1 = self.plan_segment(start, pre_grasp)
        if segment1 is None:
            return None, False
        path_segments.extend(segment1)

        # 2. □
        descend = [pre_grasp, grasp_point]
        path_segments.extend(descend)

        # 3. □

        # 4. □
        segment2 = self.plan_segment(grasp_point, post_grasp)
        if segment2 is None:
            return None, False
        path_segments.extend(segment2)

        # 5. □
        smoothed_path = self.smooth_path(path_segments)

        return smoothed_path, True

    def plan_segment(self, start, goal):
        """□□"""
        # RRT*□
        path = self.rrt_planner.plan(start, goal, max_time=2.0)

        if path is None:
            # RRT□
            path = self.rrt_planner.plan(start, goal, max_time=1.0)

        return path
```

```
    def smooth_path(self, path):
        """□□"""
        smoother = PathSmoother()

        # B□
        smoothed = smoother.b_spline_smooth(path, num_points=len(path) * 3)

        # □
        smoothed = smoother.shortcut_smoothing(smoothed, self.obstacles)

        return smoothed

    def validate_path(self, path):
        """□□"""
        for point in path:
            # □
            for obs in self.obstacles:
                if self.is_point_in_obstacle(point, obs):
                    return False

            # □orkspace□
            if not self.is_in_workspace(point):
                return False

        return True

    def is_point_in_obstacle(self, point, obstacle):
        """□□"""
        # □
        center = obstacle['center']
        radius = obstacle['radius']
        dist = np.linalg.norm(np.array(point) - np.array(center))
        return dist < radius

    def is_in_workspace(self, point):
        """□□"""
        for i, (val, (low, high)) in enumerate(zip(point, self.workspace)):
            if val < low or val > high:
                return False
        return True
```

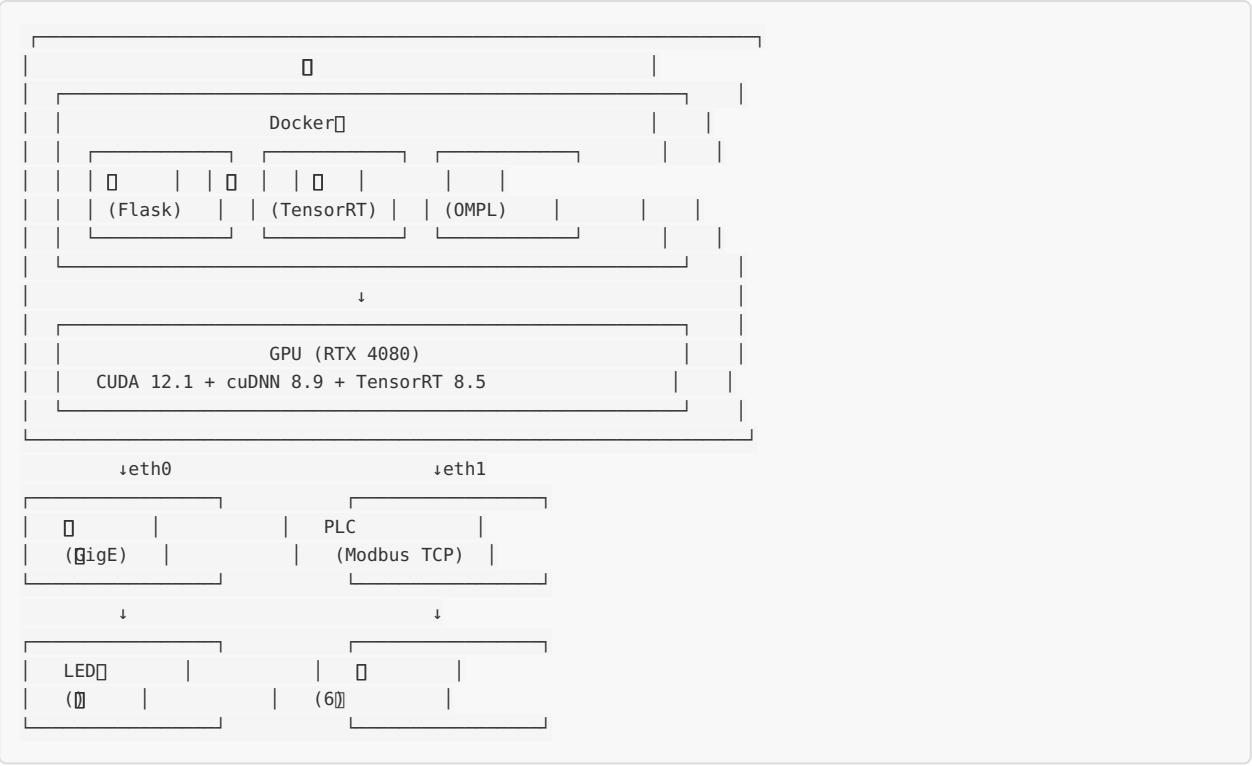
---

7. □□□□□□□

7.1 □□□□

```yaml
# system_config.yaml
system:
  name: SteelPlateDetection
  version: 1.0.0
  log_level: INFO

camera:
  type: GigE
  resolution: [2448, 2048]
  fps: 30
  exposure: 10000  # us
  gain: 1.0

lighting:
  type: LED
  intensity: 80  # %
  polarization_angle: 90  # □

detection:
  model_path: models/yolov8_defect.engine
  conf_threshold: 0.5
  iou_threshold: 0.4
  device: cuda:0

edge:
  method: adaptive_canny
  morph_iterations: 2
  skeletonize: true
  subpixel_refinement: true

grasp:
  offset_from_edge: 50  # mm
  approach_height: 100  # mm
  max_grasp_force: 500  # N

plc:
  ip: 192.168.1.100
  port: 502
  slave_id: 1
  register_defect_status: 40001
  register_grasp_command: 40100

performance:
  target_fps: 30
  max_detection_time: 50  # ms
  max_path_planning_time: 500  # ms
```

7.2 □□□□


□
Docker□
□
(Flask)
□
(TensorRT)
□
(OMPL)
GPU (RTX 4080)
CUDA 12.1 + cuDNN 8.9 + TensorRT 8.5
↓eth0
↓eth1
□
(□igE)
PLC
(Modbus TCP)
LED□
(□)
□
(6□)

7.3 □□□□□

□□□	□□	□□
□□□□	□□□□□□□□□	□□□□□□
□□□□	□□□□□□□□□	□□□□□□
□□□□	□□□□□□□□□	□□□□
□□□□	□□□□□□□□□□□	□□□□
□□□□	□□□□□□	□□□□

8. 性能指标与测试

8.1 测试指标体系

指标类别	指标项	目标值	测试方法
检测精度	缺陷检出率	≥98%	人工标注对比
	误检率	≤3%	人工复核
	分类准确率	≥95%	混淆矩阵
定位精度	边缘定位精度	≤0.2mm	标定板测量
	缺陷位置精度	≤1mm	标定板测量
实时性	单帧处理时间	≤50ms	计时统计
	路径规划时间	≤500ms	计时统计
稳定性	连续运行时间	≥24h	压力测试
	MTBF	≥1000h	可靠性测试

8.2 □□□□□□

```
 import time
import numpy as np

class PerformanceBenchmark:
    """□□"""

    def __init__(self, detection_pipeline, edge_locator, grasp_planner):
        self.detector = detection_pipeline
        self.edge_locator = edge_locator
        self.grasp_planner = grasp_planner

    def benchmark_detection(self, test_images, num_runs=100):
        """□□"""
        times = []

        for _ in range(num_runs):
            img = test_images[np.random.randint(0, len(test_images))]

            start = time.time()
            detections = self.detector.detect(img)
            elapsed = (time.time() - start) * 1000  # ms

            times.append(elapsed)

        return {
            'mean': np.mean(times),
            'std': np.std(times),
            'p50': np.percentile(times, 50),
            'p95': np.percentile(times, 95),
            'p99': np.percentile(times, 99),
        }

    def benchmark_edge_localization(self, test_images):
        """□□"""
        times = []

        for img in test_images:
            start = time.time()
            edges, lines = self.edge_locator.locate_edge(img)
            elapsed = (time.time() - start) * 1000
            times.append(elapsed)

        return {
            'mean': np.mean(times),
            'std': np.std(times),
        }

    def benchmark_path_planning(self, num_tests=100):
        """□□"""
        times = []
        success_count = 0

        for _ in range(num_tests):
            # □
            start = [np.random.randint(0, 1000) for _ in range(3)]
            goal = [np.random.randint(0, 1000) for _ in range(3)]

            start_time = time.time()
            path, success = self.grasp_planner.plan_grasp_path(start, goal)
            elapsed = (time.time() - start_time) * 1000

            times.append(elapsed)
            if success:
                success_count += 1

        return {
```

```
            'mean_time': np.mean(times),
            'success_rate': success_count / num_tests * 100,
        }
```

---

9. 维护与升级

9.1 日常维护

维护项	周期	操作
相机清洁	每日	软布擦拭镜头
光源检查	每日	检查亮度、偏振片
标定验证	每周	标定板验证精度
模型更新	按需	新数据重新训练
系统备份	每周	配置、模型备份

9.2 模型更新流程

```
1. 
        ↓
2. 
        ↓
3. 
        ↓
4. 
        ↓
5. A/B
        ↓
6. 
        ↓
7. 
```

---

10. 生成式人工智能与视觉检测的未来趋势

10.1 从判别式到生成式的范式转变

传统计算机视觉以判别式模型为主导——从ImageNet分类到目标检测，核心范式是学习 $P(Y|X)$：给定输入图像，输出预定义的类别标签。这种"观察→分类"的模式在标准化任务中表现优异，但面临三个根本性局限：

判别式模型的三大局限：

局限	具体表现	对缺陷检测的影响
分布外泛化弱	仅学习条件边界，新缺陷类型泛化差	未知缺陷漏检率高
数据饥渴	依赖大规模标注，罕见缺陷样本稀缺	工业场景数据获取成本高
不确定性缺失	输出硬标签，无法估计置信度	关键场景误检风险高

[illegible]

[illegible] $P(X, Y)$ [illegible]"[illegible]"[illegible]"[illegible]"[illegible]

1. [illegible] [illegible] Diffusion Model [illegible]"[illegible]→[illegible]"[illegible]
2. [illegible] [illegible]
3. [illegible] [illegible]

10.2 Diffusion Model[illegible]

[illegible]

[illegible]

```
class DiffusionDefectDetection:
    """
    □

    □
             □
    """

    def __init__(self, model_path, threshold=0.1):
        self.model = DiffusionModel(model_path)
        self.threshold = threshold

    def detect_anomaly(self, image):
        """
        □

        Args:
            image: □(H, W, 3)

        Returns:
            anomaly_score: □
            is_defect: □
        """
        # 1. □
        z0 = self.model.encode(image)

        # 2. □
        noisy_z = self.add_noise(z0, t=500)

        # 3. □
        reconstructed = self.model.denoise(noisy_z, t=0)

        # 4. □
        error = np.linalg.norm(z0 - reconstructed)

        # 5. □
        anomaly_score = error / (np.linalg.norm(z0) + 1e-6)

        return anomaly_score, anomaly_score > self.threshold

    def add_noise(self, z, t, T=1000):
        """□"""
        alpha_t = self.cosine_schedule(t / T)
        noise = np.random.randn(*z.shape)
        return np.sqrt(alpha_t) * z + np.sqrt(1 - alpha_t) * noise
```

[illegible]

[illegible]	[illegible]	[illegible]
[illegible]	DreamSim[illegible]	[illegible]40%
[illegible]	[illegible]	[illegible]60%
[illegible]	Inpainting[illegible]	[illegible]25%
[illegible]	[illegible]	[illegible]70%

10.3 [illegible]

Anti-Interference-2D[illegible]

[illegible]

```
┌──────────────────────────────────────────────────────┐
│                     [illegible]                      │
├──────────────────────────────────────────────────────┤
│                                                      │
│  ┌────────────────┐          ┌────────────────┐      │
│  │  [illegible]   │          │  [illegible]   │      │
│  │  (YOLOv8)      │          │  ([illegible]) │      │
│  │                │          │                │      │
│  │  • [illegible] │          │  • [illegible] │      │
│  │  • [illegible] │          │  • [illegible] │      │
│  │  • [illegible] │          │  • [illegible] │      │
│  └───────┬────────┘          └───────┬────────┘      │
│          │                           │               │
│          └─────────────┬─────────────┘               │
│                        ↓                             │
│               ┌───────────────┐                      │
│               │  [illegible]  │                      │
│               │               │                      │
│               │  [illegible]  │                      │
│               │  [illegible]  │                      │
│               │  [illegible]  │                      │
│               └───────────────┘                      │
│                                                      │
└──────────────────────────────────────────────────────┘
```

[illegible]

```
class HybridDefectSystem:
    """□□"""

    def __init__(self, detector, generator, fusion_weight=0.7):
        self.detector = detector          # YOLOv8□
        self.generator = generator        # □
        self.fusion_weight = fusion_weight  # □

    def detect(self, image):
        """
        □

        Args:
            image: □

        Returns:
            results: □
        """
        # 1. □
        disc_results = self.discriminative_detect(image)

        # 2. □
        gen_scores = self.generative_score(image)

        # 3. □
        fused_results = []
        for det in disc_results:
            # □
            region_score = self.get_region_score(det['bbox'], gen_scores)

            # □
            fused_conf = (self.fusion_weight * det['conf'] +
                          (1 - self.fusion_weight) * (1 - region_score))

            det['fused_conf'] = fused_conf

            # □
            if fused_conf > 0.3 or region_score > 0.7:
                fused_results.append(det)

        return fused_results

    def generative_score(self, image):
        """□□"""
        # □
        anomaly_map = self.generator.compute_anomaly_map(image)
        return anomaly_map
```

---

11. □□□□□□□□□□

11.1 □□□□□□□□□□□

□□□□□+HDR□□□□□□□□□□□□□□□□□□2D□□□□□□□□□

局限性	具体表现	影响
深度信息缺失	2D图像无深度维度	凹坑/凸起深度无法量化
穿透能力弱	无法检测近表面缺陷	内部缺陷漏检
依赖光照	无光照则无法成像	暗区检测困难
反光敏感	强反光仍可能干扰	极端角度失效

11.2 多模态融合技术路线

可见光 + 红外热成像：

红外热成像对高反光不敏感，能检测温度异常区域：

融合方式	原理	适用场景
特征级融合	红外特征与可见光特征拼接	缺陷分类增强
决策级融合	红外检测结果与可见光结果投票	可靠性提升
时空融合	时序热图分析温度变化趋势	疲劳损伤检测

```
class MultiModalFusion:
    """□□"""

    def __init__(self, visible_model, infrared_model):
        self.visible_model = visible_model      # □
        self.infrared_model = infrared_model    # □

    def fused_detect(self, visible_img, infrared_img):
        """
        □

        Args:
            visible_img: □
            infrared_img: □

        Returns:
            fused_results: □
        """
        # 1. □
        visible_results = self.visible_model.detect(visible_img)

        # 2. □
        infrared_results = self.infrared_model.detect(infrared_img)

        # 3. □
        visible_features = self.extract_features(visible_img)
        infrared_features = self.extract_thermal_features(infrared_img)
        fused_features = np.concatenate([visible_features, infrared_features], axis=-1)

        # 4. □
        fused_results = self.decision_fusion(
            visible_results,
            infrared_results,
            fused_features
        )

        return fused_results

    def decision_fusion(self, vis_results, ir_results, fused_features):
        """□□"""
        # 1□
        # 2□
        # 3□
        # 4□

        # □
        fused = []
        all_dets = {**vis_results, **ir_results}

        for det_id, det in all_dets.items():
            vis_conf = vis_results.get(det_id, {}).get('conf', 0)
            ir_conf = ir_results.get(det_id, {}).get('conf', 0)

            # □
            fused_conf = 0.6 * vis_conf + 0.4 * ir_conf

            if fused_conf > 0.5:
                det['fused_conf'] = fused_conf
                fused.append(det)

        return fused
```

2D□□ + □□□□□

□□□□□□□□□□3D□□□□□□□□□□□□□

技术	分辨率	优势	局限
结构光	0.05-0.1mm	高精度、快速	受强光干扰
TOF	1-5mm	抗干扰、远距离	精度受限
激光三角测量	0.01-0.05mm	超高精度	需扫描移动
双目立体视觉	0.1-1mm	成本低	依赖纹理

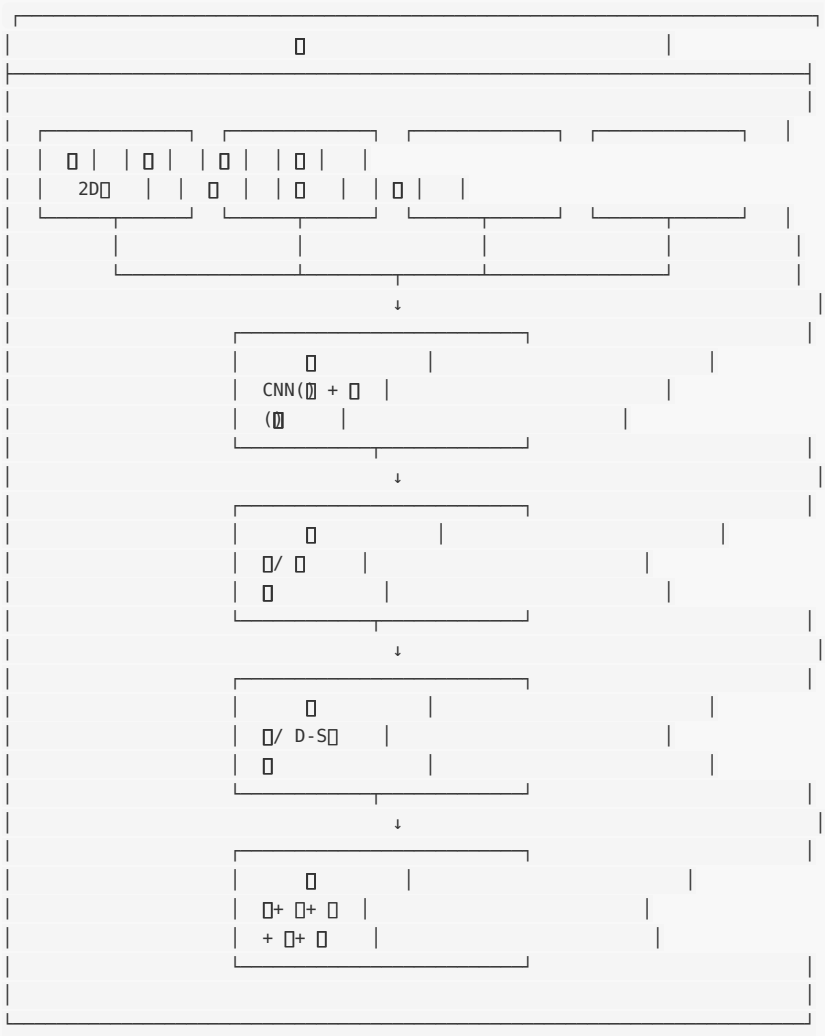
视觉 + 超声/涡流：

对近表面缺陷敏感，与视觉互补：

```
class MultiSensorSteelInspector:
    """□□"""

    def __init__(self):
        self.visible_camera = VisibleCamera()      # □
        self.polarized_camera = PolarizedCamera()  # □
        self.infrared_camera = InfraredCamera()     # □
        self.eddy_current = EddyCurrentSensor()    # □
        self.laser_scanner = LaserScanner()        # □

    def comprehensive_inspect(self, region):
        """
        □

        Args:
            region: □

        Returns:
            inspection_report: □
        """
        # 1. □
        polarized_data = self.polarized_camera.capture(region)
        surface_defects = self.analyze_surface(polarized_data)

        # 2. □
        thermal_data = self.infrared_camera.capture(region)
        thermal_anomalies = self.analyze_thermal(thermal_data)

        # 3. 3D□
        point_cloud = self.laser_scanner.scan(region)
        depth_map = self.process_3d_data(point_cloud)

        # 4. □
        eddy_data = self.eddy_current.scan(region)
        subsurface_defects = self.analyze_eddy_current(eddy_data)

        # 5. □
        report = self.fuse_results(
            surface_defects,
            thermal_anomalies,
            depth_map,
            subsurface_defects
        )

        return report

    def fuse_results(self, surface, thermal, depth, subsurface):
        """□□"""
        # □
        # 1. □
        # 2. □
        # 3. 3D□
        # 4. □

        fused_defects = []

        # □
        for defect in surface:
            defect['source'] = 'polarized_visible'
            defect['confidence'] *= 1.0
            fused_defects.append(defect)

        # □
        for thermal_defect in thermal:
            if not self.is_covered(thermal_defect, fused_defects):
                thermal_defect['source'] = 'infrared'
```

```
                thermal_defect['confidence'] *= 0.8
                fused_defects.append(thermal_defect)

        # □
        for sub_defect in subsurface:
            sub_defect['source'] = 'eddy_current'
            sub_defect['depth_category'] = 'subsurface'
            fused_defects.append(sub_defect)

        return fused_defects
```

11.3 □□□□□□□□



---

12. □□□□□□□□□□□□

12.1 Anti-Interference-2D□□□□□□

□□□□□□□□□ □□□□ + YOLOv8
- □□□□□□□□□□□□
- YOLOv8□□□□□□

- RRT*路径规划
- 单机版部署

第二阶段（**6-12**个月）：偏振光 + HDR + 生成式增强
- HDR多曝光融合提升动态范围
- Diffusion Model数据增强
- 混合范式缺陷检测
- 边缘计算部署

第三阶段（**1-2**年）：多模态融合 + 3D感知
- 增加红外热成像
- 集成激光3D扫描
- 多传感器融合决策
- 云边协同架构

第四阶段（**2-3**年）：端到端智能系统
- 实时3D缺陷重建
- 生成式缺陷模拟训练
- 自主学习持续优化
- 全自动化产线集成

12.2 行业技术趋势

趋势	技术方向	预期影响
端侧**AI**加速	NPU/TPU边缘部署	延迟<10ms，实时性大幅提升
多模态大模型	Vision-Language模型	自然语言交互检测报告
数字孪生	虚实映射 + 仿真训练	零成本算法验证
自监督学习	无标注数据利用	标注成本降低80%
量子视觉	量子计算加速	超大规模优化问题

12.3 船舶行业应用前景

沪东中华实践启示：

沪东中华造船首次将3D机器视觉用于钢板缺陷检测，实现船舶行业首例应用[40†]。这一案例表明：

1. 行业需求迫切：大型船舶企业对自动化检测接受度高
2. 技术可行性验证：3D视觉在船舶场景已落地
3. 标准引领重要性：行业标杆企业带动整体升级

Anti-Interference-2D项目定位：

本项目聚焦高反光这一细分难点，填补行业技术空白：

维度	定位
技术深度	偏振光成像 + 深度学习专项突破
应用广度	船舶/钢结构/ aerospace多元拓展
商业模式	软硬一体 + SaaS服务

13. 参考资源汇总

13.1 开源项目

项目	地址	说明
YOLOv8	https://github.com/ultralytics/ultralytics	实时目标检测
TensorRT	https://github.com/NVIDIA/TensorRT	GPU推理加速
OpenCV	https://github.com/opencv/opencv	计算机视觉基础库
Stable Diffusion	https://github.com/AUTOMATIC1111/stable-diffusion-webui	扩散模型生成

13.2 学术论文

论文	引用	核心贡献
DreamSim (2024)	[47†]	生成式异常检测新范式
YOLOv8 (2023)	[72†]	实时目标检测SOTA
HiDiff (2024)	[具体引用]	扩散模型辅助分割
Faster R-CNN (2015)	[98†]	两阶段检测基准

13.3 行业标准

标准	说明
GB/T 3323-2005	钢板表面缺陷等级
ISO 5817	焊接缺陷分类
CCS	中国船级社检测规范

文档版本: v2.0
更新日期: 2026-04-27
项目名称: Anti-Interference-2D 大型船舶钢板面高反光工况2D视觉检测
来源: 整合高反光钢板视觉检测调研报告、代码实现指南及图形学与几何学融合理论